

Prospect App - Developer Implementation Guide

Quick Reference: From 45 Broken Tabs to 6 Working Modules

The Transformation

BEFORE: 45+ Broken Tabs	→	AFTER: 6 Working Modules
├ 5 Training Systems	→	├ Dashboard Hub
├ 5 Team Visualizations	→	├ Smart CRM
├ 4 Resource Libraries	→	├ AI Content Studio
├ 3 Achievement Systems	→	├ AI Business Coach
├ 2 Content Calendars	→	├ Team Collaboration
├ 20+ Duplicate Features	→	├ Business Tracker
└ 80% Non-functional		

Module-by-Module Implementation

Module 1: Dashboard Hub

File Structure:

```
/src/modules/dashboard/  
├ components/  
|   ├── DailyPriorities.tsx  
|   ├── QuickStats.tsx  
|   ├── AITip.tsx  
|   └── QuickActions.tsx  
├ hooks/  
|   ├── useDashboardData.ts  
|   └── useAIPrioritization.ts  
└ api/  
    └ dashboard.ts
```

Key Implementation:

typescript

```
// DailyPriorities.tsx
```

```
interface Priority {
  id: string;
  title: string;
  type: 'contact' | 'content' | 'follow-up';
  dueTime: string;
  importance: 'high' | 'medium' | 'low';
}

const DailyPriorities: React.FC = () => {
  const { priorities } = useAIPrioritization();

  return (
    <Card>
      <CardHeader>
        <h2>Today's Top 3 Priorities</h2>
      </CardHeader>
      <CardContent>
        {priorities.slice(0, 3).map(priority => (
          <PriorityItem key={priority.id} {...priority} />
        ))}
      </CardContent>
    </Card>
  );
};
```

API Endpoints:

```
GET /api/dashboard/priorities
GET /api/dashboard/stats
GET /api/dashboard/ai-tip
POST /api/dashboard/quick-action
```

Module 2: Smart CRM

File Structure:

```

/src/modules/crm/
├── components/
│   ├── ContactList.tsx
│   ├── ContactForm.tsx
│   ├── Pipeline.tsx
│   ├── LeadScoring.tsx
│   └── FollowUpReminders.tsx
├── hooks/
│   ├── useContacts.ts
│   ├── usePipeline.ts
│   └── useLeadScoring.ts
└── api/
    ├── contacts.ts
    └── pipeline.ts

```

Key Implementation:

typescript

// Pipeline.tsx with Drag & Drop

```
import { DndContext, DragEndEvent } from '@dnd-kit/core';
```

```

const Pipeline: React.FC = () => {
  const { stages, moveContact } = usePipeline();

  const handleDragEnd = (event: DragEndEvent) => {
    const { active, over } = event;
    if (over) {
      moveContact(active.id, over.id);
    }
  };

  return (
    <DndContext onDragEnd={handleDragEnd}>
      <div className="flex gap-4 overflow-x-auto">
        {stages.map(stage => (
          <PipelineStage key={stage.id} stage={stage} />
        ))}
      </div>
    </DndContext>
  );
};

```

Database Schema:

```
sql

-- Contacts with AI scoring
CREATE TABLE contacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255),
  phone VARCHAR(50),
  stage VARCHAR(50) DEFAULT 'new',
  score INTEGER DEFAULT 50,
  tags TEXT[],
  custom_fields JSONB,
  last_contact TIMESTAMP,
  next_followup TIMESTAMP,
  notes JSONB DEFAULT '[]',
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Pipeline stages
CREATE TABLE pipeline_stages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  name VARCHAR(100),
  order_index INTEGER,
  color VARCHAR(7),
  rules JSONB
);
```

Module 3: AI Content Studio

File Structure:

```
/src/modules/content/  
├─ components/  
│   ├─ ContentGenerator.tsx  
│   ├─ AvatarVideoCreator.tsx  
│   ├─ ImageGenerator.tsx  
│   ├─ ContentCalendar.tsx  
│   └─ TemplateLibrary.tsx  
├─ hooks/  
│   ├─ useContentGeneration.ts  
│   ├─ useAvatarVideo.ts  
│   └─ useImageGeneration.ts  
└─ api/  
    ├─ openai.ts  
    ├─ did.ts  
    └─ content.ts
```

OpenAI Integration:

typescript

```

// openai.ts
import { Configuration, OpenAIApi } from 'openai';

const configuration = new Configuration({
  apiKey: process.env.OPENAI_API_KEY,
});

const openai = new OpenAIApi(configuration);

export async function generateContent({
  type,
  platform,
  topic,
  tone,
  includeProduct
}: ContentRequest): Promise<ContentResponse> {
  const prompt = buildPrompt({ type, platform, topic, tone, includeProduct });

  const completion = await openai.createChatCompletion({
    model: "gpt-4",
    messages: [
      { role: "system", content: getSystemPrompt(type) },
      { role: "user", content: prompt }
    ],
    temperature: 0.7,
    max_tokens: 500
  });

  const content = completion.data.choices[0].message.content;

  // Add platform-specific optimizations
  return optimizeForPlatform(content, platform);
}

// Image Generation
export async function generateImage(prompt: string): Promise<string> {
  const response = await openai.createImage({
    prompt,
    n: 1,
    size: "1024x1024",
    response_format: "url"
  });
}

```

```
    return response.data.data[0].url;
  }
```

D-ID Avatar Integration:

typescript

// did.ts

```
export async function createAvatarVideo({
  script,
  presenter = 'amy',
  voice = 'en-US-AriaNeural'
}: AvatarRequest): Promise<AvatarResponse> {
  const response = await fetch('https://api.d-id.com/talks', {
    method: 'POST',
    headers: {
      'Authorization': `Basic ${process.env.DID_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      script: {
        type: 'text',
        input: script
      },
      config: {
        fluent: true,
        pad_audio: 0.0
      },
      source_url: `https://d-id.com/presenters/${presenter}.jpg`,
      driver_url: 'bank://lively'
    })
  });

  return response.json();
}
```

Module 4: AI Business Coach

File Structure:


```
/src/modules/coach/  
├ components/  
|   ├── ChatInterface.tsx  
|   ├── ObjectionHandler.tsx  
|   ├── RolePlaySimulator.tsx  
|   ├── VoiceCoach.tsx  
|   └─ ScriptLibrary.tsx  
├ hooks/  
|   ├── useCoaching.ts  
|   ├── useVoiceAnalysis.ts  
|   └─ useScripts.ts  
└─ api/  
    ├── coaching.ts  
    └─ elevenlabs.ts
```

Coaching Implementation:

typescript

```
// ChatInterface.tsx
```

```
const ChatInterface: React.FC = () => {  
  const [messages, setMessages] = useState<Message[]>([]);  
  const { sendMessage, isLoading } = useCoaching();  
  
  const handleSend = async (content: string) => {  
    const userMessage = { role: 'user', content };  
    setMessages(prev => [...prev, userMessage]);  
  
    const response = await sendMessage(content);  
    setMessages(prev => [...prev, { role: 'assistant', content: response }]);  
  };  
  
  return (  
    <div className="flex flex-col h-full">  
      <MessageList messages={messages} />  
      <MessageInput onSend={handleSend} disabled={isLoading} />  
    </div>  
  );  
};
```

```
// Coaching Prompts
```

```
const coachingPrompts = {  
  objections: {  
    "too_expensive": `  
      When someone says "It's too expensive," here are 3 proven responses:  
      1. "I understand price is important. What specifically about the investment concerns you?"  
      2. "Many of my best customers said the same thing initially. Can I share what changed the  
      3. "If price wasn't a factor, would this be something you'd want to do?"  
    `,  
    "pyramid_scheme": `  
      This is a common concern. Here's how to address it professionally:  
      1. Explain the difference: "I appreciate your concern. Let me show you how our business m  
      2. Focus on products: "We're a product-driven company. Our income comes from actual sales  
      3. Share compliance: "We follow all FTC guidelines and have been in business for X years.  
    `,  
  },  
  
  daily: {  
    morning: "What are your three main goals for today? Let's make them specific and achievable  
    midday: "How's your energy level? Let's do a quick check-in on your progress.",  
    evening: "What went well today? What will you do differently tomorrow?"  
  }  
};
```

```
}  
};
```

Voice Analysis Integration:

typescript

```
// elevenlabs.ts  
export async function analyzeVoice(audioBlob: Blob): Promise<VoiceAnalysis> {  
  // Convert audio to text first  
  const transcript = await transcribeAudio(audioBlob);  
  
  // Analyze voice characteristics  
  const analysis = {  
    pace: calculatePace(audioBlob),  
    tone: analyzeTone(audioBlob),  
    confidence: measureConfidence(transcript, audioBlob),  
    clarity: assessClarity(audioBlob)  
  };  
  
  // Generate feedback  
  const feedback = await generateVoiceFeedback(analysis);  
  
  return { transcript, analysis, feedback };  
}
```

Module 5: Team Collaboration

File Structure:

```
/src/modules/team/  
├─ components/  
|   ├─ TeamDirectory.tsx  
|   ├─ ResourceLibrary.tsx  
|   ├─ Challenges.tsx  
|   ├─ EventCalendar.tsx  
|   └─ Recognition.tsx  
├─ hooks/  
|   ├─ useTeam.ts  
|   ├─ useChallenges.ts  
|   └─ useEvents.ts  
└─ api/  
    ├─ team.ts  
    └─ challenges.ts
```

Team Implementation:

typescript

```
// Team Creation and Management
```

```
interface Team {  
  id: string;  
  name: string;  
  code: string; // 6-digit invite code  
  members: TeamMember[];  
  resources: Resource[];  
  challenges: Challenge[];  
}
```

```
const createTeam = async (name: string): Promise<Team> => {  
  const code = generateInviteCode();  
  
  const team = await supabase  
    .from('teams')  
    .insert({  
      name,  
      code,  
      created_by: user.id  
    })  
    .single();  
  
  return team;  
};
```

```
// Challenge System
```

```
const ChallengeCard: React.FC<{ challenge: Challenge }> = ({ challenge }) => {  
  const { joinChallenge, updateProgress } = useChallenges();  
  
  return (  
    <Card>  
      <CardHeader>  
        <h3>{challenge.title}</h3>  
        <Badge>{challenge.duration} days</Badge>  
      </CardHeader>  
      <CardContent>  
        <ProgressBar value={challenge.userProgress} max={challenge.target} />  
        <div className="mt-4 space-y-2">  
          <Button onClick={() => updateProgress(challenge.id)}>  
            Log Activity  
          </Button>  
        </div>  
      </CardContent>  
    </Card>  
  );  
};
```

```
    </Card>
  );
};
```

Module 6: Business Tracker

File Structure:

```
/src/modules/tracker/
├── components/
│   ├── IncomeTracker.tsx
│   ├── ActivityLog.tsx
│   ├── GoalProgress.tsx
│   ├── HabitTracker.tsx
│   └── Journal.tsx
├── hooks/
│   ├── useTracking.ts
│   ├── useGoals.ts
│   └── useHabits.ts
└── api/
    ├── tracking.ts
    └── ocr.ts
```

Income Tracking with OCR:

typescript

```

// IncomeTracker.tsx
import Tesseract from 'tesseract.js';

const IncomeTracker: React.FC = () => {
  const { addIncome } = useTracking();

  const handleImageUpload = async (file: File) => {
    // Show loading state
    setProcessing(true);

    // Process image with OCR
    const { data: { text } } = await Tesseract.recognize(file, 'eng');

    // Extract numbers from text
    const amounts = extractAmounts(text);

    // Show confirmation dialog
    const confirmed = await confirmAmounts(amounts);

    if (confirmed) {
      await addIncome(amounts);
    }

    setProcessing(false);
  };

  return (
    <div className="space-y-4">
      <Card>
        <CardHeader>
          <h2>Track Your Income</h2>
        </CardHeader>
        <CardContent>
          <div className="grid grid-cols-2 gap-4">
            <Button onClick={() => setShowManualEntry(true)}>
              Manual Entry
            </Button>
            <label className="btn btn-secondary">
              Upload Statement
            <input
              type="file"
              accept="image/*"
              onChange={(e) => handleImageUpload(e.target.files[0])}
            </input>
          </div>
        </CardContent>
      </Card>
    </div>
  );
};

```

```
        className="hidden"
      />
    </label>
  </div>
</CardContent>
</Card>

<IncomeChart data={incomeData} />
<GoalProgress current={totalIncome} target={monthlyGoal} />
</div>
);
};
```

Technical Specifications

Frontend Stack

```
json
{
  "mobile": {
    "framework": "React Native",
    "navigation": "React Navigation v6",
    "state": "Zustand + React Query",
    "ui": "NativeBase + Custom Components",
    "offline": "WatermelonDB"
  },
  "web": {
    "framework": "Next.js 14",
    "ui": "Tailwind CSS + Shadcn/ui",
    "state": "Zustand + React Query",
    "auth": "NextAuth.js"
  }
}
```

Backend Architecture

yaml

api:

framework: Express.js + TypeScript

orm: Prisma

validation: Zod

services:

- name: core-api

routes:

- /auth
- /users
- /contacts
- /content
- /team
- /tracking

- name: ai-service

framework: FastAPI

routes:

- /generate
- /coach
- /analyze

database:

primary: PostgreSQL (Supabase)

cache: Redis

storage: S3

integrations:

- OpenAI API
- D-ID API
- ElevenLabs API
- Twilio API
- WhatsApp Business API

Mobile Optimization

typescript

// Progressive Web App Configuration

```
const pwaConfig = {
  name: 'Prospect',
  short_name: 'Prospect',
  theme_color: '#6366F1',
  background_color: '#111827',
  display: 'standalone',
  orientation: 'portrait',
  icons: [
    { src: '/icon-192.png', sizes: '192x192' },
    { src: '/icon-512.png', sizes: '512x512' }
  ]
};
```

// Offline Capabilities

```
const offlineConfig = {
  contacts: 'full_sync',
  content: 'drafts_only',
  activities: 'queue_for_sync',
  ai: 'cached_responses'
};
```

Migration Plan

Phase 1: Data Migration (Week 1)

```
sql

-- Migrate existing data to new schema
INSERT INTO contacts_new (name, email, phone, user_id)
SELECT name, email, phone, user_id
FROM contacts_old
WHERE email IS NOT NULL;

-- Map old features to new modules
UPDATE users SET
  preferences = jsonb_set(
    preferences,
    '{migrated}',
    'true'
  );
```

Phase 2: Feature Parity (Week 2)

- Implement working versions of currently broken features
- Ensure no data loss during transition
- Maintain user sessions

Phase 3: Gradual Rollout (Week 3)

- 10% of users on new version
 - Monitor metrics and feedback
 - Fix critical issues
 - Increase to 50%, then 100%
-

Cost-Benefit Analysis

Current State Costs

- Development time wasted: 6+ months
- User churn from frustration: 70%+
- Support tickets: 100+ per day
- Reputation damage: Immeasurable

Redesign Investment

- Development: 8 weeks $\times$ \$10k = \$80k
- APIs and services: \$2k/month
- Infrastructure: \$1k/month
- Total first year: ~\$116k

Expected Returns

- User retention improvement: 70% $\rightarrow$ 30% churn
- Conversion rate: 10% $\rightarrow$ 30% free-to-paid
- Support reduction: 100 $\rightarrow$ 20 tickets/day
- Monthly recurring revenue: \$50k $\rightarrow$ \$200k

ROI Timeline

- Break-even: Month 3

- Positive ROI: Month 6
 - 5x return: Year 1
-

Launch Checklist

Pre-Launch

- ☐ All 6 modules functional
- ☐ Mobile apps approved (iOS/Android)
- ☐ API integrations tested
- ☐ Performance optimized (<2s load)
- ☐ Security audit passed
- ☐ Documentation complete
- ☐ Video tutorials recorded
- ☐ Support team trained

Launch Day

- ☐ Migration scripts ready
- ☐ Monitoring active
- ☐ Support channels open
- ☐ Marketing campaign live
- ☐ Rollback plan ready
- ☐ Team on standby

Post-Launch

- ☐ Daily metrics review
 - ☐ User feedback collection
 - ☐ Bug fix priority queue
 - ☐ Feature request tracking
 - ☐ Performance monitoring
 - ☐ Scaling adjustments
-

Success Criteria

Week 1 Post-Launch

- 80% of users successfully migrated
- <5% increase in support tickets

- Core features working for 95%+ users
- No critical bugs

Month 1

- 70% daily active users
- 25% free-to-paid conversion
- 4.0+ app store rating
- <5% monthly churn

Month 3

- 1,000+ paying customers
 - \$50k+ MRR
 - 50+ NPS score
 - Feature requests > bug reports
-

Resources

Documentation

- [API Documentation](#)
- [Component Library](#)
- [Database Schema](#)
- [Integration Guides](#)

Training

- Developer onboarding video
- Architecture overview
- Best practices guide
- Troubleshooting manual

Support

- Slack channel: #prospect-dev
- Weekly sync meetings
- On-call rotation
- Escalation procedures

Remember: The goal is to build a simple, working app that provides real value to network marketers every single day. No broken promises, no placeholder features, just tools that work.